

WatchLockAI Sentinel - Prioritized TODO

Generated: 2025-09-03T03:02:21Z

This document provides a human-readable view of prioritized work items for WatchLockAI Sentinel. Items are categorized by priority (P0 = Critical, P1 = Important, P2 = Enhancement) and estimated development effort.

P0 - Critical Issues (Must Fix)

These items represent critical issues that affect system stability, security, or core functionality. All P0 items should be addressed before production deployment.

P0-001: Fix Windows Registry Monitor Import Guards PLATFORM_GUARD

Files: `collectors/reg_monitor.py`

Effort: 30 minutes

Risk: Low

Platform guards implemented but may have typing issues with conditional imports. Need to add proper type stubs for `winreg` when module not available on Linux.

Acceptance Criteria:

- `pyright=0` on `collectors/reg_monitor.py`
 - Registry monitor loads without errors on both Windows and Linux
-

P0-002: Fix Windows Service Wrapper Import Guards

PLATFORM_GUARD

Files: `service/service_wrapper.py`

Effort: 45 minutes

Risk: Low

Platform guards for win32 modules may have typing issues with conditional imports. Add proper type stubs for win32 modules when not available on Linux.

Acceptance Criteria:

- pyright=0 on `service/service_wrapper.py`
 - Service wrapper loads without errors on Linux
 - Service functions return appropriate error messages on Linux
-

P0-003: Fix Response Actions Platform-Specific Code

PLATFORM_GUARD

Files: `response/actions.py`

Effort: 25 minutes

Risk: Medium

Platform-aware timeouts and error handling may have typing inconsistencies. Ensure proper typing for platform-specific timeout and error message logic.

Acceptance Criteria:

- pyright=0 on `response/actions.py`
 - Process termination works correctly on both platforms
 - Appropriate timeouts applied per platform
-

P0-004: Missing Console Web API Import Guard

IMPORT_CYCLE

Files: `console/web_api.py`, `service/service_wrapper.py`

Effort: 20 minutes

Risk: Medium

Service wrapper imports `console.web_api` but has `try/except` guard that may not be typed properly. Need proper typing and lazy import pattern for optional web API dependency.

Acceptance Criteria:

- `pyright=0` on `service/service_wrapper.py`
 - Service starts correctly when FastAPI not available
 - Graceful degradation message logged
-

P0-005: Event Schema Type Union Resolution

EVENT_SCHEMA

Files: `app_core/schemas.py`

Effort: 60 minutes

Risk: High

`SentinelEvent` union type may need explicit type discrimination for proper routing. Add explicit discriminator field or improve union type handling.

Acceptance Criteria:

- `pyright=0` on `app_core/schemas.py`
 - Event bus routing works correctly for all event types
 - No runtime type resolution errors
-

P1 - Important Issues (Should Fix)

These items improve system reliability, performance, and maintainability. Address these after P0 items are resolved.

P1-001: Add Missing Error Handling in Event Bus Worker

EVENT_SCHEMA

Files: `app_core/bus.py`

Effort: 90 minutes

Risk: Low

Event bus worker has basic error handling but could be more robust for edge cases. Add comprehensive error handling and recovery for event delivery failures.

P1-002: Implement Configuration Hot Reload CONFIG

Files: `app_core/config.py`

Effort: 3 hours

Risk: Medium

Configuration currently requires restart to reload changes from config.yaml. Add file watcher for config.yaml changes and safe hot reload mechanism.

P1-003: Add Comprehensive API Input Validation

API_ROUTER

Files: `console/web_api.py`, `console/api/operational_mode.py`

Effort: 2 hours

Risk: Medium

API endpoints have basic validation but could be more comprehensive. Add comprehensive input validation, rate limiting, and error response formatting.

P1-004: Optimize Event Bus Performance for High Load

EVENT_SCHEMA

Files: `app_core/bus.py`

Effort: 4 hours

Risk: Low

Event bus uses `asyncio.Queue` which may not be optimal for very high event volumes. Implement event batching and optimize queue performance for high throughput.

P1-005: Add Threat Intelligence Database Caching

CONFIG

Files: `detection/threat_intel_db.py`

Effort: 2.5 hours

Risk: Low

Threat intelligence searches may be slow without proper caching. Implement LRU cache for threat intelligence queries with TTL expiration.

P2 - Enhancement Issues (Nice to Have)

These items add new features and capabilities. Implement after core system is stable and P0/P1 items are addressed.

P2-001: Add Behavioral Engine Machine Learning Integration DOCS

Files: `detection/behavioral_engine.py`

Effort: 8 hours

Risk: High

Behavioral engine exists but lacks ML-based anomaly detection. Integrate scikit-learn or similar for behavioral anomaly detection.

P2-002: Implement Advanced File Quarantine System

CONFIG

Files: `response/actions.py`

Effort: 6 hours

Risk: High

Dependencies: P0-003

Build secure file quarantine with restore capability and audit trail. Files can be quarantined to secure location with restoration capability.

P2-003: Add Web Console Authentication API_ROUTER

Files: `console/web_api.py`

Effort: 5 hours

Risk: Medium

Dependencies: P1-003

Web console currently has no authentication (localhost only). Implement session-based authentication for web console access.

P2-004: Implement Real-time Dashboard Updates API_ROUTER

Files: `console/web_api.py`

Effort: 4 hours

Risk: Medium

Dependencies: P1-001

Dashboard requires manual refresh for updated information. Add WebSocket or SSE for real-time dashboard updates.

P2-005: Add Comprehensive Integration Tests TEST

Files: `tests/`

Effort: 6.5 hours

Risk: Low

Limited integration tests for end-to-end workflows. Build comprehensive integration test suite covering major workflows.

P2-006: Implement Configuration Schema Validation CONFIG

Files: `app_core/config.py`

Effort: 3 hours

Risk: Low

Dependencies: P1-002

Configuration validation relies on Pydantic but lacks comprehensive schema validation. Add JSON schema validation for configuration.

P2-007: Add Process Tree Analysis EVENT_SCHEMA

Files: `collectors/proc_monitor.py`, `detection/behavioral_engine.py`

Effort: 5.5 hours

Risk: Medium

Dependencies: P1-004

Process monitoring captures parent PID but doesn't build process tree analysis. Implement process tree construction and suspicious pattern detection.

P2-008: Implement Memory-Resident Threat Hunting

DOCS

Files: `detection/`

Effort: 10 hours

Risk: High

Dependencies: P1-001, P1-005

System focuses on real-time detection but lacks historical threat hunting. Add capability to hunt for threats in historical event data.

Summary Statistics

By Priority

- **P0 (Critical):** 5 items, ~3 hours total effort
- **P1 (Important):** 5 items, ~12.5 hours total effort
- **P2 (Enhancement):** 8 items, ~51 hours total effort

By Category

- **PLATFORM_GUARD:** 3 items (all P0)

- **EVENT_SCHEMA:** 4 items (1 P0, 2 P1, 1 P2)
- **CONFIG:** 4 items (1 P1, 3 P2)
- **API_ROUTER:** 3 items (1 P1, 2 P2)
- **IMPORT_CYCLE:** 1 item (P0)
- **TEST:** 1 item (P2)
- **DOCS:** 2 items (P2)

By Risk Level

- **High Risk:** 5 items (requires careful implementation)
- **Medium Risk:** 6 items (moderate complexity)
- **Low Risk:** 7 items (straightforward fixes)

Recommended Implementation Order

1. **Phase 1** - Fix Critical Platform Issues (P0-001, P0-002, P0-003)
2. **Phase 2** - Resolve Import and Type Issues (P0-004, P0-005)
3. **Phase 3** - Improve System Reliability (P1-001, P1-002)
4. **Phase 4** - Enhance API and Performance (P1-003, P1-004, P1-005)
5. **Phase 5** - Add Advanced Features (Selected P2 items based on requirements)

This prioritization ensures core system stability before adding enhancements, with platform compatibility as the highest priority.